MEMORIA DEL PROYECTO

Gestión de una asociación de juegos de mesa

Aplicación Java con JDBC y MariaDB



Lorena Arbé Sánchez
Aitor Amaury Farinango Rojas

149FA - 3.º DAM
Módulo: Acceso a Datos
Curso 2026–2027

Índice

Índice....2
1. Introducción....3
2. Puesta en marcha....3
2.1. Requisitos....3
2.2. Preparar la base de datos....3
2.3. Ejecutar la aplicación y las pruebas....3
2.4. Repositorio....3
3. Funcionalidades principales....4
4. Organización del código y patrones....4
Patrones utilizados....4
5. Decisiones de diseño....4
6. Modelo entidad-relación y relaciones....5
6.1. Relaciones y justificación....5
6.2. Integridad referencial y borrados....6
7. Pruebas....6
8. Actas de seguimiento....7
Acta 1 · Preparación y diseño....7
Acta 2 · Acceso a datos y funcionalidades....7
Acta 3 · Revisión, pruebas y documentación....8
Planificación del proyecto....8
9. Conclusiones....9

1. Introducción

Aplicación de consola desarrollada en Java para gestionar una asociación de juegos de mesa de Vitoria-Gasteiz. Utiliza MariaDB para guardar la información y JDBC para conectar con la base de datos.

2. Puesta en marcha

2.1. Requisitos

- Java JDK compatible con el proyecto.
- IntelliJ IDEA (IDE utilizado).
- MariaDB o MySQL y phpMyAdmin/XAMPP para preparar la base de datos.
- Dependencias Maven, incluido el conector JDBC de MariaDB y las librerías utilizadas para exportar.

2.2. Preparar la base de datos

1. Iniciar MariaDB desde XAMPP y abrir phpMyAdmin.
2. Importar el fichero juegos.sql del repositorio. El script crea la estructura y contiene los datos iniciales incluidos en él.
3. Comprobar que existe la base de datos juegos y que se han creado sus tablas y relaciones.
4. Revisar la URL, el puerto, el usuario y la contraseña de la conexión JDBC. En el entorno de desarrollo se ha utilizado el puerto 3307; adaptar este dato si se ejecuta en otra instalación.

2.3. Ejecutar la aplicación y las pruebas

- Abrir el proyecto en IntelliJ IDEA y esperar a que Maven cargue las dependencias.
- Ejecutar App.java mediante su método main() para abrir el menú de consola.
- Para las pruebas, iniciar MariaDB y ejecutar AppTest.java con JUnit desde IntelliJ.

2.4. Repositorio

Repositorio público: https://github.com/Lorena-Arbe-Sanchez/Repo_Juegos_Mesa

- **Proyecto**: código Java.
- **MER**: imágenes y archivos del modelo entidad-relación.
- **juegos.sql**: script de creación/preparación de la base de datos.
- **README.md**: descripción, tecnologías, funcionalidades y puesta en marcha.
- **LICENSE**: licencia del repositorio.

Clonado del repositorio:

```
git clone https://github.com/Lorena-Arbe-Sanchez/Repo_Juegos_Mesa.git
```

3. Funcionalidades principales

- Gestión de la asociación: consulta y modificación de sus datos.
- Juegos: alta, listado, consulta, modificación, eliminación y búsquedas por nombre, temática y número de jugadores.
- Socias: alta, listado, consulta por documento, modificación, eliminación y búsquedas por nombre y apellidos.
- Invitadas: gestión diferenciada de las personas que participan en partidas sin ser socias.
- Partidas: alta, listado, consulta, modificación, eliminación y búsquedas por juego o fecha.
- Exportación de juegos, socias y partidas a XML y JSON.
- Consulta y exportación de metadatos de la base de datos.

4. Organización del código y patrones

- **models**: clases que representan los datos (Juego, Socia, Invitada, Partida y Asociación).
- **idao**: interfaces que declaran las operaciones de acceso a datos.
- **dao**: implementaciones JDBC con SQL, PreparedStatement y ResultSet.
- **utils**: utilidades compartidas, como DatabaseConnection.
- **App.java**: menú y punto de entrada de la aplicación.

Patrones utilizados

DAO (Data Access Object): separa las consultas y operaciones SQL del menú y del resto de la aplicación. Cada DAO agrupa las operaciones de una entidad.

Singleton: DatabaseConnection centraliza el acceso a la conexión JDBC mediante getInstance(), evitando repetir su configuración en cada DAO.

5. Decisiones de diseño

- MariaDB y JDBC puro: permiten trabajar directamente con SQL y conexiones, tal como pide el módulo.
- Socias e invitadas separadas: las socias pertenecen a la asociación; las invitadas pueden participar sin ser socias.
- Documento como clave: socia e invitada se identifican mediante DNI/NIE/Pasaporte.
- Propiedad de juegos: un juego puede pertenecer a la asociación o a una socia; se representa con asociacion_id o socia_documento.
- Participantes: se utilizan tablas intermedias distintas para socias e invitadas.
- Ganadoras: es_ganadora en las tablas intermedias indica qué participantes ganaron la partida.
- Exportación XML/JSON y metadatos: facilitan el intercambio de datos y la consulta de la estructura de la BBDD.

6. Modelo entidad-relación y relaciones

Tablas principales y su función:

Tabla	Clave primaria	Contenido
asociacion	id	Datos de la asociación y contacto.
juego	id	Características, estado y propietaria del juego.
socia	documento	Datos personales y de contacto de las socias.
invitada	documento	Datos personales y de contacto de las invitadas.
partida	id	Juego, fecha, hora y plazas.
partida_socia	partida_id + socia_documento	Participación de socias y marca de ganadora.
partida_invitada	partida_id + invitada_documento	Participación de invitadas y marca de ganadora.

6.1. Relaciones y justificación

- Asociación–Juego: una asociación puede tener varios juegos. asociacion_id permite registrar los juegos que pertenecen a la asociación.
- Socia–Juego: una socia puede ser propietaria de varios juegos. socia_documento identifica a la propietaria.
- Juego–Partida: un juego puede utilizarse en varias partidas; cada partida referencia un juego mediante juego_id.
- Partida–Socia: relación muchos a muchos resuelta con partida_socia.
- Partida–Invitada: relación muchos a muchos resuelta con partida_invitada. Las dos tablas intermedias guardan también es_ganadora.

En juego hay dos referencias opcionales (asociacion_id y socia_documento) para representar los dos tipos de propietaria. La regla de que solo una esté informada debe controlarse desde la aplicación o reforzarse con una restricción en la base de datos.

6.2. Integridad referencial y borrados

Las claves foráneas evitan referencias a registros inexistentes. Según las acciones ON DELETE del script SQL, algunos borrados se propagan en cascada: al borrar una partida se eliminan sus filas de participación; al borrar un juego se eliminan sus partidas y, por esa relación, las participaciones dependientes. En las relaciones que no tienen CASCADE, la base de datos impide borrar el registro principal mientras existan referencias. Los DAO ejecutan DELETE y MariaDB aplica las reglas; si el borrado no está permitido, se produce una SQLException. Este comportamiento debe coincidir con las restricciones del juegos.sql entregado.

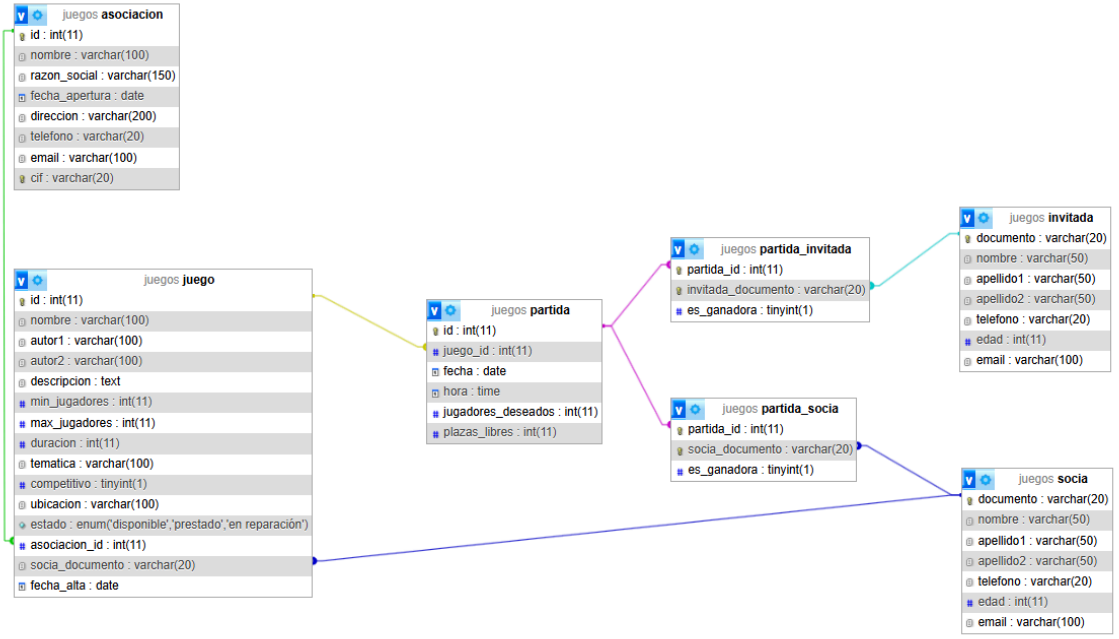


Figura 1. Modelo entidad-relación de la base de datos juegos generada en phpMyAdmin.

7. Pruebas

Se ha creado una suite JUnit en AppTest.java. En la última ejecución realizada, los seis tests finalizaron correctamente. Se comprueban la conexión y operaciones principales de los DAO: altas, consultas, listados, modificaciones y eliminaciones. Para repetirlas, la base de datos debe estar iniciada y preparada con juegos.sql.

8. Actas de seguimiento

Las siguientes actas resumen el avance de las tres semanas a partir de las fechas, responsables, estados e hitos de la planificación compartida. Se han redactado como resumen semanal del trabajo registrado; no se dispone de fechas concretas de reuniones.

Acta 1 · Preparación y diseño

Periodo: 12–18 de octubre de 2026

Participantes: Lorena Arbé Sánchez y Aitor Amaury Farinango Rojas

Avances:

- 12/10: creación del proyecto en IntelliJ IDEA y del repositorio GitHub.
- 13–14/10: creación y codificación de las clases de modelo.
- 13–15/10: elaboración de varias versiones del MER y creación de las tablas e inserción de datos en phpMyAdmin.
- 14/10: revisión conjunta del código y del MER; puesta en común de criterios.
- 14/10: inicio del trabajo de la rama DAO.

Decisiones / incidencias: Se definió la estructura inicial de la aplicación y el modelo de datos antes de avanzar con las operaciones JDBC.

Objetivos siguientes: Continuar con las implementaciones DAO, las pruebas y las primeras funcionalidades del menú.

Acta 2 · Acceso a datos y funcionalidades

Periodo: 19–25 de octubre de 2026

Participantes: Lorena Arbé Sánchez y Aitor Amaury Farinango Rojas

Avances:

- Continuación del desarrollo de las clases DAO.
- 18/10: implementación del listado y exportación de metadatos.
- 18/10: implementación de exportaciones XML y JSON.
- 19/10: desarrollo del menú de App.java.
- 20–25/10: implementación de las operaciones CRUD del menú.
- 21/10: inicio de búsquedas avanzadas y control de errores.

Decisiones / incidencias: Se avanzó desde la estructura de datos hacia las operaciones disponibles desde el menú. Se mantuvo el reparto de tareas indicado en la planificación.

Objetivos siguientes: Completar búsquedas y manejo de errores, terminar los DAO, ampliar las pruebas y preparar la interfaz/despliegue.

Acta 3 · Revisión, pruebas y documentación

Periodo: 26 de octubre–1 de noviembre de 2026

Participantes: Lorena Arbé Sánchez y Aitor Amaury Farinango Rojas

Avances:

- Hasta el 28/10: continuación y cierre del hito de desarrollo de los DAO.
- 31/10: finalización de las búsquedas avanzadas.
- 1/11: finalización del control de errores y de la tarea de interfaz reflejada en la planificación.
- 1/11: finalización de la suite JUnit; los seis tests ejecutados finalizaron correctamente.
- Desde el 28/10: elaboración del documento de memoria del proyecto.

Decisiones / incidencias:

- Se revisaron las funcionalidades terminadas y se comprobó el funcionamiento de los casos incluidos en la suite.
- **Integración externa (BGG)**: tras investigar las condiciones de acceso a su API, se decidió no implementar esta funcionalidad. BGG requiere el uso de tokens en las peticiones y, tras crear una cuenta, es necesario esperar a que aprueben el acceso, un proceso que puede tardar más de una semana. Debido a estas limitaciones y al tiempo disponible, se consideró que no era eficiente incorporarla en esta fase del proyecto.
- Las tareas de análisis/estadísticas y seguridad/control aparecen todavía en curso en la planificación.

Planificación del proyecto

La captura siguiente muestra las tareas, prioridades, responsables, estados y fechas utilizadas como base para el seguimiento semanal.

Tareas_del_proyecto

	Tarea	Prioridad	Propietario	Estado	Fecha de inicio	Fecha de finalización	Hito	Distribuible	Notas
2	Crear proyecto en IntelliJ y repositorio inicial	P0	LORENA ARB...	Completada	12/10/2026	12/10/2026	Tener proyecto y repo	Archivo	Enlace repo
3	Crear y codificar ficheros de clases en IntelliJ	P0	LORENA ARB...	Completada	13/10/2026	14/10/2026	Clases funcionales	Archivo	Notas
4	Hacer MER (varias versiones)	P1	LORENA ARB...	Completada	13/10/2026	15/10/2026	MER de referencia	Archivo	Notas
5	Crear tablas en phpMyAdmin e insertarles datos	P0	LORENA ARB...	Completada	13/10/2026	15/10/2026	Tablas en phpMyAdmin	Archivo	Notas
6	Revisar todo el código actual y el MER + Feedback conjunto	P1	AITOR AMAU...	Completada	14/10/2026	14/10/2026	Puesta en común	Apuntes proye...	Notas
7	Crear rama e ir codificando los ficheros de las carpetas DAO	P1	AITOR AMAU...	Completada	14/10/2026	28/10/2026	Hito	Archivo	Notas
8	Hacer suite de tests / pruebas en fichero AppTest.java (JUnit)	P2	LORENA ARB...	Completada	15/10/2026	1/11/2026	Calidad del código	Archivo	Notas
9	Implementar listado y exportación de los metadatos	P1	LORENA ARB...	Completada	18/10/2026	18/10/2026	Exportación de metadatos	Archivo	Notas
10	Implementar exportación a XML y JSON	P1	LORENA ARB...	Completada	18/10/2026	18/10/2026	Exportación a XML y JSON	Archivo	Notas
11	Ir codificando el menú de la clase App.java	P1	LORENA ARB...	Completada	19/10/2026	1/11/2026	Hito	Archivo	Notas
12	Ir codificando las funciones del menú (CRUD)	P1	AITOR AMAU...	Completada	20/10/2026	25/10/2026	Hito	Archivo	Notas
13	Implementar búsquedas avanzadas	P2	LORENA ARB...	Completada	21/10/2026	31/10/2026	Búsquedas avanzadas	Archivo	Notas
14	Implementar control de errores exhaustivo en todo el código	P1	LORENA ARB...	Completada	21/10/2026	1/11/2026	Control de errores	Archivo	Notas
15	Implementar interfaz y despliegue	P2	AITOR AMAU...	Completada	30/10/2026	1/11/2026	Interfaz y despliegue	Archivo	Notas
16	Implementar análisis y estadísticas	P2	Nombre	No iniciada	d/mm/yyyy	d/mm/yyyy	Análisis y estadísticas	Archivo	Notas
17	Implementar seguridad y control	P2	AITOR AMAU...	En curso	31/10/2026	1/11/2026	Seguridad y control	Archivo	Notas
18	Implementar integración externa	P2	AITOR AMAU...	Bloqueada	d/mm/yyyy	d/mm/yyyy	Integración externa	Archivo	Tras informarnos, concluimos que no era ef
19	Documento de Memoria / Manual	P3	LORENA ARB...	Completada	28/10/2026	1/11/2026	Memoria del proyecto	Archivo	Notas

9. Conclusiones

El proyecto integra Java, JDBC y MariaDB para gestionar juegos, personas y partidas. La estructura por modelos y DAO organiza el código; las claves foráneas y tablas intermedias mantienen las relaciones; y las exportaciones, búsquedas, metadatos y pruebas amplían las funciones básicas de gestión.